Project Introduction

This hotel reservation system is designed to manage room bookings and user interactions for both guests and administrators. The system provides a user-friendly interface for guests to log in, view available rooms, make reservations, and manage their bookings. Administrators can manage room availability, view all reservations, and cancel bookings as needed.

Built using object-oriented programming principles, the system incorporates key OOP pillars such as Encapsulation, Inheritance, and Polymorphism to ensure modularity, maintainability, and flexibility. The system also includes features like dynamic room generation and user authentication, enhancing both the user experience and the management capabilities.

Report Table of Contents

I. The Roles and Relationships of Classes in the Project

II. Detailed Introduction of Each Class

1)Room Class

2)RoomGenerator Class

3)LoginSystem Class

4)User Class

5)PanelOperation Class (Abstract Class)

6)AdminPanel (Subclass)

7)GuestPanel Class (Subclass)

8)MainMenu Class

III. Characteristics of the Project

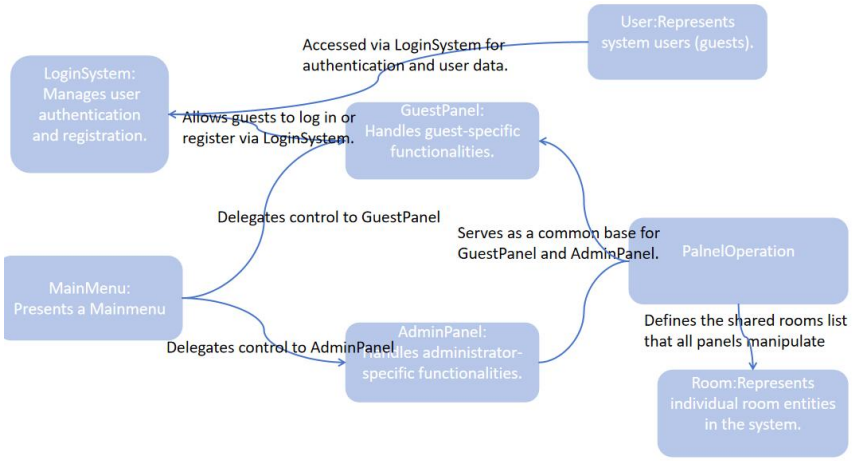
1)OOP Pillars in My Hotel Reservation System

2)Guest Login System

3)Color Printing in the Project

4)Robustness in My Project

I.The Roles and Relationships of Classes in the Project



1.MainMenu: Central controller and program entry point.Initializes RoomGenerator and LoginSystem.Provides a menu to switch between GuestPanel, AdminPanel, or exit.
2.GuestPanel: Handles guest operations like logging in, booking, and managing reservations.
Relies on LoginSystem for authentication and uses the shared room list.
3.AdminPanel: Manages administrative tasks such as adding/removing rooms and overseeing reservations.Uses LoginSystem for managing users and reservations.
4.PanelOperation: Abstract class shared by GuestPanel and AdminPanel.Provides common methods for room and reservation management.
5.Room: Represents individual rooms with attributes like type, price, and availability.Links to User to associate bookings.
6.User: Stores user credentials and reservations.Manages interactions with the Room class.
7.LoginSystem: Handles user login, registration, and management.Supports both GuestPanel and AdminPanel.
8.RoomGenerator: Generates initial room data with predefined or random attributes.Used during system setup by MainMenu.
Summary: MainMenu initializes the system, delegating control to GuestPanel or AdminPanel. Both panels inherit shared functionality from PanelOperation. LoginSystem manages users, while Room and User handle reservations. RoomGenerator provides initial room data for the system.

II. Detailed Introduction of Each Class

1)Room Class

1. Class Purpose

The Room class models a hotel room with attributes such as room type, category, availability, and pricing. It provides functionality for managing room booking details, availability status, and calculating the cost of a stay.

2. Attributes

Enums :
(1)**RoomType**:Represents the type of room.Possible values: SINGLE, DOUBLE, SUITE.
(2)**Category**:Represents the category of the room.Possible values: DELUXE, SUITE, STANDARD.
Fields:
(1)**roomNumber** (int):The unique identifier for the room.
(2)**roomType** (RoomType):Specifies the type of the room (e.g., SINGLE, DOUBLE, SUITE).
(3)**category** (Category):Specifies the category of the room (e.g., DELUXE, SUITE, STANDARD).
(4)**available** (boolean):Indicates whether the room is available for booking.
(5)**price** (double):The price per night for the room. This value is calculated based on the roomType.
(6)**nights** (int):Tracks the number of nights a room is booked.
(7)**user** (User):A reference to the user who booked the room.

3. Constructor

Room(int roomNumber, Category category, RoomType roomType)
Initializes a room with:
(1)roomNumber: Unique identifier for the room.
(2)category: The category of the room.
(3)roomType: The type of the room.
(4)Sets available to true by default, indicating the room is available for booking.
(5)Computes the base price using the calculatePrice method.

4.Key Methods

private double calculatePrice(): Determines the base price of the room based on its type:
1.SINGLE: RMB 500 2.DOUBLE: RMB 800 3.SUITE: RMB 1200
Returns 0 by default (should not occur unless an invalid type is provided).
public double getFinalPrice(): Computes the total cost of a booking considering:Discounts for stays of 7 more nights (20% discount).
Additional multipliers based on the room category:DELUXE: 3x base price. SUITE: 1.5x base price. STANDARD: Base price.
public double getPerNightPrice(): Computes the price per night based on the room category:DELUXE: 3x base price. SUITE: 1.5x base price. STANDARD: Base price.
public boolean isAvailable(): Checks if the room is currently available.
public void setNights(int nights): Sets the number of nights for which the room is booked.
public int getNights():Returns the number of nights the room is booked.
public void setAvailable(boolean available):Updates the availability status of the room.
public void setUser(User user):Associates a user with the room when it is booked.
public String getUserName():Retrieves the username of the user who booked the room.
public int getRoomNumber():Returns the unique room number.
public Category getCategory():Retrieves the room's category.
public RoomType getRoomType():Retrieves the room's type.
public String toString():Provides a string representation of the room's details:
Room number, category, type, price, availability, booking user, and reserved nights.
Displays "Booked by: None" if the room is not booked.

2)RoomGenerator Class

Purpose

The RoomGenerator class automates the creation of a list of Room objects with randomized categories and types. This helps to populate the hotel system with a diverse set of rooms during initialization.

Attributes

This class does not have any attributes or instance variables, as it relies on static methods and local variables for its functionality.

Methods

1.generateRooms()

Purpose:Generates a list of Room objects with randomly assigned attributes.

Return Type:List<Room>: A list containing the generated Room objects.

Details

(1)Initialization:Creates an empty list of rooms (ArrayList).

Instantiates a Random object to generate random integers.

(2)Room Generation Logic:The method uses a loop to generate a fixed number of rooms (5 by default, but it can be adjusted).

(3)For each room:1.A random number is generated to determine the category of the room (DELUXE, SUITE, or STANDARD).2.Another random number is used to assign the type of the room (SINGLE, DOUBLE, or SUITE).3.A Room object is created using these randomly selected attributes and a unique room number (i).4.The room is added to the list.

(4)Room Attributes Assignment:

Category:If the random number is 0, the category is set to DELUXE.If the random number is 1, the category is set to SUITE.Otherwise, the category is set to STANDARD.

Type:If the random number is 0, the type is set to SUITE.If the random number is 1, the type is set to DOUBLE.Otherwise, the type is set to SINGLE.

(5)Return:Returns the list of Room objects after the loop completes.

3) LoginSystem Class

Purpose

The LoginSystem class manages user authentication and registration. It stores user credentials and ensures secure access to the system.

Attributes

(1)**ANSI_RESET and ANSI_LIGHT_CYAN (String)**:

Constants for ANSI color codes, used to style console outputs.

(2)**users (Map<String, User>)**:A static map that associates usernames (String) with User objects.

Represents the database of registered users in the system.

Constructor

LoginSystem(): Initializes the users map.Adds two sample users (user1 and user2) for testing purposes, each with a username and password.

Key Methods

login(): Handles user login by:
1.Prompting the user for their username and password via the console.
2.Validating the credentials against the users map.
3.Returning the corresponding User object if the credentials are valid.
4.Printing a success message for valid credentials or an error message for invalid credentials.
Algorithm:
1.Fetch the User object using the entered username.
2.Compare the entered password with the stored password.
3.Return the User object on success or null on failure.
register(): Handles user registration by:
1.Prompting the user for a unique username and password.
2.Checking if the username already exists in the users map.
3.Creating a new User object and adding it to the users map if the username is available.
4.Printing a success or error message based on the outcome.
Algorithm:
1.Check if the entered username exists in users.
2.If it exists, prompt the user to try again.
3.Otherwise, add the new user to the users map.
getUsers(): Returns all registered users as a Collection<User>.

(4)User Class

Purpose

The User class models a system user by encapsulating their credentials (username and password) and maintaining a list of their room reservations. It serves as the core entity for managing user-specific data in the hotel reservation system.

Attributes

1. **username (String)**:The unique identifier for the user.
2. **password (String)**:The password for the user account, used for authentication.
3. **reservations (List<Room>)**:A list of room objects representing the user's reservations.Usage:
This list is dynamically updated when the user adds a new reservation or cancels an existing one.

Constructor

User(String username, String password):
Purpose:Initializes a new User object with the provided username and password.
Initialization:username and password are set from the parameters.
reservations is initialized as an empty ArrayList.
Example Usage:User user = new User("john_doe", "securePassword");

Methods

1. addReservation(Room room): Purpose:Adds a Room object to the user's reservations list.
Details:The method takes a Room object as input.Adds the room to the reservations list.
2. getReservations(): Purpose:Retrieves the list of rooms reserved by the user.
Return Type:A List<Room> containing the user's reservations.
3. getUsername(): Purpose:Retrieves the username of the user.Return Type:String
4. getPassword(): Purpose:Retrieves the password of the user.Return Type:String

5) PanelOperation Class (Abstract Class)

Purpose

PanelOperation serves as an abstract base class for defining a common interface for panel operations, such as managing rooms and reservations. Subclasses like AdminPanel and GuestPanel inherit from it and implement its methods.

Key Features

1.**Centralized Room List**:The rooms attribute is a static List<Room> that acts as a shared repository for all room data.
All subclasses manipulate this shared list.
2.Abstract Design:Contains method definitions (placeholders) for common operations such as adding rooms, viewing rooms, booking, and canceling reservations.
Subclasses must override these methods to provide specific implementations.
3.Encapsulation of Core Operations:Defines operations like addRoom(), removeRoom(), viewAvailableRooms(), and cancelReservationGuest(User user) but does not implement them.

Methods

addRoom():Adds a room to the system. Implementation is left to subclasses.
removeRoom(LoginSystem loginSystem):Removes a room and handles associated reservations.
viewAllRooms() and viewAvailableRooms():Display all rooms or only available ones.
bookARoom(User user):Allows a user to book a room.
checkReservations(LoginSystem L):Displays all user reservations.
cancelReservationAdmin(LoginSystem L):Allows an admin to cancel a reservation.
cancelReservationGuest(User user):Allows a guest to cancel their own reservation.
getRooms():Returns the list of rooms managed by the system.

6).AdminPanel (Subclass)

Purpose

The AdminPanel class extends PanelOperation and provides implementations of operations specifically for administrators. It focuses on managing rooms and user reservations.

Key Features

1.Room Management:Adds, removes, and views all rooms, including detailed information such as availability and price.
Ensures that added rooms have unique numbers.
2.Reservation Management:Allows administrators to view all reservations in the system.
Can cancel any user's reservation, freeing up associated rooms.
3.**Integration with LoginSystem**:Uses LoginSystem to manage users and their reservations.

Methods

1. addRoom():Allows administrators to add rooms with specified categories, types, and unique room numbers.
Ensures the entered room number is unique by checking the rooms list.
2. removeRoom(LoginSystem loginSystem):Removes a specified room and cancels all associated reservations for that room.
Updates the rooms list and modifies user reservations using the LoginSystem.
3. viewAllRooms():Displays detailed information about all rooms, including their availability, type, price, and the user who booked them (if any).
4. checkReservations(LoginSystem L):Lists all reservations in the system, grouped by users.
5. cancelReservationAdmin(LoginSystem L):Enables the admin to cancel a specific user's reservation by selecting from a list of reservations.
6. main():Provides a menu-driven interface for administrators to perform operations like adding, viewing, and removing rooms, as well as managing reservations.

7) GuestPanel Class(Subclass)

Purpose

The GuestPanel class extends PanelOperation and provides operations specifically for guests, such as viewing available rooms, booking rooms, and managing their reservations.

Key Features

1.Booking System: Allows guests to view and book available rooms.
Handles room availability and updates the user's reservation list.
2.Reservation Management: Enables guests to view their current reservations.
Provides functionality for canceling their reservations.
3.Login and Registration: Integrates with the LoginSystem to handle guest login and registration before accessing the panel.

Methods

1. viewAvailableRooms(): Displays all rooms that are currently available for booking.
Uses the isAvailable() method of the Room class to filter results.
2. bookARoom(User user): Allows a guest to book a specific room for a given number of nights.
Updates the room's availability, associates the user with the room, and adds the room to the

user's reservations.
3. ViewYourReservations(User user): Displays all rooms currently reserved by the logged-in guest.
4. cancelReservationGuest(User user): Enables a guest to cancel one of their reservations.
Updates the room's availability and removes it from the user's reservation list.
5. logMenu(LoginSystem L): Provides a login and registration interface for guests.
Ensures that only authenticated users can access the GuestPanel.
6. main(): Offers a menu-driven interface for guests to interact with the panel.
Allows actions like viewing available rooms, booking, and canceling reservations.

8)MainMenu Class

Purpose

The MainMenu class initializes the system, sets up the required components, and presents a menu-driven interface to allow users to choose between the Guest Panel, the Admin Panel, or exit the program.

Valuables

1.ANSI_RESET and ANSI_LIGHT_CYAN: Constants used for formatting console output with ANSI escape codes.
ANSI_RESET resets the text color to default.
ANSI_LIGHT_CYAN changes the text color to light cyan for better visibility.
2.PanelOperation A:A polymorphic reference of type PanelOperation, allowing the system to dynamically switch between AdminPanel and GuestPanel.
3.LoginSystem L: An instance of the LoginSystem class, used for handling user authentication and registration.
4.generatedRooms:A list of Room objects generated using the RoomGenerator utility. These rooms are added to the shared room list in PanelOperation.
5.Scanner input:A Scanner object for reading user input from the console.

Methods

1. main(String[] args):

Purpose
The main method initializes the system and provides a menu-driven interface for selecting between the Guest Panel and Admin Panel, or exiting the program.
Implementation
1.Room Generation:Calls RoomGenerator.generateRooms() to create a list of rooms.
Adds the generated rooms to the shared rooms list using an AdminPanel instance.
2.Login System Initialization:Creates an instance of LoginSystem to handle user authentication.
3.Menu Interface:Displays a menu with three options:
(1)Guest Panel: Launches the GuestPanel where guests can book rooms or manage their reservations.

(2)Admin Panel: Launches the AdminPanel where administrators can manage rooms and user reservations.

(3)Exit: Exits the program.

4.User Interaction:Reads the user's choice using the Scanner object and performs the corresponding action:

If 1 is selected, it initializes the GuestPanel and calls its main() method.

If 2 is selected, it initializes the AdminPanel and calls its main() method.

If 3 is selected, it exits the program after displaying a thank-you message.

For invalid input, prompts the user to try again.

III.Characteristics of the project

1)OOP Pillars in My Hotel Reservation System

1. Encapsulation: Data like `roomNumber` in the `Room` class and `username` in the `User` class are private, accessible only through public methods (e.g., `getRoomNumber()` or `addReservation()`).

This ensures data integrity and prevents unauthorized manipulation, such as modifying room availability or user credentials directly.

2. Inheritance: The abstract class `PanelOperation` serves as a base for `GuestPanel` and `AdminPanel`, providing shared methods like `viewAllRooms()` and `bookARoom()`.

Subclasses extend and implement specific behaviors, such as admins adding rooms or guests booking them.

3. Polymorphism: The `PanelOperation` reference in `MainMenu` dynamically invokes methods on `GuestPanel` or `AdminPanel` at runtime.

Subclasses override methods to customize functionality, like guests viewing their bookings versus admins managing all reservations.

```
switch (choice) {
    case 1:
        System.out.println(ANSI_LIGHT_CYAN+"Launching Guest Panel..."+ANSI_RESET);
        // Create a new GuestPanel and start its main method
        A = new GuestPanel();
        GuestPanel.main(args, A, L);
        break;
    case 2:
        System.out.println(ANSI_LIGHT_CYAN+"Launching Admin Panel..."+ANSI_RESET);
        // Create a new AdminPanel and start its main method
        A = new AdminPanel();
        AdminPanel.main(args, A, L);
        break;
```

Summary: Encapsulation safeguards data, inheritance reduces redundancy, and polymorphism ensures flexibility. Together, these principles make my system modular, maintainable, and extensible.

2)Guest Login System

The LoginSystem class manages user authentication and registration in the project:
Login: Users provide a username and password. If the credentials match an existing user, they are successfully logged in; otherwise, an error message is displayed.
Registration: Users can create an account by providing a unique username and password. If the username already exists, they are prompted to try again.
This system ensures secure user access and is used by the GuestPanel for authentication, while the AdminPanel accesses the stored user data for managing reservations.

```
Welcome to the Guest Login System
1. Login
2. Register
3. Exit
Please enter your choice: 1
Enter username: 123
Enter password:
```

3)Color Printing in the Project

My project uses ANSI escape codes for adding color to console outputs:
Constants:ANSI_LIGHT_CYAN: For light cyan text. ANSI_RESET: Resets text to the default color.
Usage: Highlights prompts, success messages, and errors (e.g., "Login successful!" or "Invalid choice").Applied in LoginSystem, GuestPanel, and AdminPanel to emphasize menus, room details, and feedback.
Benefits: Improves readability and user experience by making important information visually distinct.
Adds a professional and engaging interface for users.

4)Robustness in My Project

My project ensures robust performance by:
Error Handling: Validates inputs (e.g., room availability, unique usernames) and provides clear feedback for invalid actions.
Edge Cases: Prevents booking unavailable rooms or accessing nonexistent reservations, maintaining consistent updates.
Data Integrity: Protects key attributes using encapsulation and synchronizes room data with user reservations.
These features enhance system reliability and stability.

```
1. Guest Panel
2. Admin Panel
3. Exit
Please enter your choice: 4
Invalid choice, please try again.
```